

vnr-qckit

Claude Code plugin cho VNR Playwright QC automation.

Mục lục

- [Cài đặt](#)
- [Cấu trúc plugin](#)
- [Tổng quan kiến trúc](#)
- [Các skill và lệnh](#)
- [Luồng thực thi theo giai đoạn](#)
- [Chi tiết từng skill](#)
 - [Giai đoạn 1 — Cài đặt & Khởi tạo](#)
 - [qc_pre](#)
 - [qc_setup](#)
 - [qc_basepage](#)
 - [Giai đoạn 2 — Khám phá Ứng dụng](#)
 - [qc_detect_component](#)
 - [qc_component_rule](#)
 - [qc_user_flow](#)
 - [Giai đoạn 3 — Mapping Page Object](#)
 - [qc_map_flow](#)
 - [qc_url_page_map](#)
 - [qc_sub-system-map](#)
 - [Giai đoạn 4 — Sinh Test Case](#)
 - [qc_generate](#)
 - [Giai đoạn 5 — Chạy Test Tự động](#)
 - [qc_auto_test](#)
 - [Giai đoạn 6 — Phân loại Kết quả](#)
 - [qc_triage](#)
- [Bảng Input/Output nhanh](#)
- [Cấu trúc thư mục output](#)
- [Yêu cầu](#)
- [Quy ước](#)

Cài đặt

Bước 1 — Thêm marketplace

```
claude plugin marketplace add <đường-dẫn-hoặc-repo>
```

Ví dụ từ local path:

```
claude plugin marketplace add D:/SourceCode/AutomationProcess/vnr-qckit
```

Ví dụ từ GitHub:

```
claude plugin marketplace add your-org/vnr-qckit
```

Bước 2 — Install plugin

```
claude plugin install vnr-qckit@vnr-qckit --scope project
```

`--scope project` → ghi vào `.claude/settings.json` của project, shared với cả team.

Bước 3 — Reload trong Claude Code

```
/reload-plugins
```

Cấu trúc plugin

```
vnr-qckit/
├── .claude-plugin/
│   ├── plugin.json          ← Manifest: tên, version, mô tả plugin
│   └── marketplace.json     ← Catalog để self-host marketplace
├── .claude/
│   └── settings.local.json  ← Permissions, hooks cục bộ
├── .mcp.json                ← Khai báo MCP server (playwright)
├── bin/
│   └── vnr-qckit.js         ← CLI entry point (npm bin)
├── hooks/
│   └── qckit-permissions.js ← Hook tự động cấp quyền tool
├── skills/                  ← Mỗi thư mục con = 1 slash command
│   ├── playwright_cli/     ← /vnr-qckit:playwright_cli
│   │   ├── SKILL.md        ← Driver browser qua MCP (internal)
│   │   └── references/     ← Tài liệu tham chiếu Playwright
│   │       ├── request-mocking.md
│   │       ├── running-code.md
│   │       ├── session-management.md
│   │       ├── store-state.md
│   │       └── test-generation.md
```

└─ tracing.md	
└─ qc_pre/ └─ SKILL.md	← /vnr-qckit:qc_pre (orchestrator)
└─ qc_setup/ └─ SKILL.md	← /vnr-qckit:qc_setup (internal)
└─ qc_basepage/ └─ SKILL.md	← /vnr-qckit:qc_basepage (internal)
└─ qc_detect_component/ └─ SKILL.md	← /vnr-qckit:qc_detect_component
└─ qc_component_rule/ └─ SKILL.md	← /vnr-qckit:qc_component_rule
└─ qc_user_flow/ └─ SKILL.md	← /vnr-qckit:qc_user_flow
└─ qc_map_flow/ └─ SKILL.md	← /vnr-qckit:qc_map_flow
└─ qc_url_page_map/ └─ SKILL.md	← /vnr-qckit:qc_url_page_map
└─ qc_sub-system-map/ └─ SKILL.md	← /vnr-qckit:qc_sub-system-map
└─ qc_generate/ └─ SKILL.md	← /vnr-qckit:qc_generate
└─ qc_auto_test/ └─ SKILL.md	← /vnr-qckit:qc_auto_test
└─ qc_triage/ └─ SKILL.md	← /vnr-qckit:qc_triage
└─ templates/ └─ basepage_template.md	← Template BasePage.ts (hardcode)
└─ template-component-rule.md	← Template format component-rule.md
└─ prompts/ └─ components.md	← Danh sách Angular component + selector hints
└─ components_core.md	← DOM HTML reference cho Kendo/legacy
└─ create_catalog.md	← Prompt sinh data catalog
└─ helper/ └─ playwright_cli.md	← Hướng dẫn dùng playwright-cli MCP
└─ package.json	← npm metadata (bin, files, engines)
└─ INSTALL.md	← Hướng dẫn cài đặt chi tiết
└─ README.md	← Tài liệu này
└─ VNR-QCKIT-FLOW.md	← Tài liệu luồng đầy đủ (format, ví dụ)

Tổng quan kiến trúc

GD 1 – CÀI ĐẶT	GD 2 – KHÁM PHÁ	GD 3 – MAPPING
----------------	-----------------	----------------

qc_pre - qc_setup - qc_basepage - url-aliases.md	qc_detect_component - crawl DOM/component - component_{alias}.md qc_component_rule - component-rule.md qc_user_flow - userflow-{group}.md	qc_map_flow {AliasPage}.ts {AliasLocator}.ts - locator-map.md qc_url_page_map - url-page-map.md qc_sub-system-map - sub-system-map.json
GD 4 – SINH TEST CASE qc_generate - testcase.md - datafake.json - knowledge/*.md	GD 5 – CHẠY TEST qc_auto_test - lọc TC Auto chưa PASS - sinh/tái dùng spec.ts - chạy Playwright - cập nhật testcase.md - HISTORY.md	GD 6 – PHÂN LOẠI qc_triage - PASS / APP_BUG / INFRA - Severity table - triage-*.md

Các skill và lệnh

Lệnh	Loại	Mô tả ngắn
/vnr-qckit:qc_pre	Orchestrator	Setup toàn bộ project: Playwright, BasePage, url-aliases.md
/vnr-qckit:qc_setup	Internal	Cài Playwright + Chromium, tạo playwright.config.ts, global-setup.ts
/vnr-qckit:qc_basepage	Internal	Tạo/cập nhật BasePage.ts từ template chuẩn
/vnr-qckit:qc_detect_component	Khám phá	Crawl DOM từng màn hình → sinh component_{alias}.md
/vnr-qckit:qc_component_rule	Khám phá	Tổng hợp selector rule → sinh/cập nhật component-rule.md
/vnr-qckit:qc_user_flow	Khám phá	Crawl action từng màn hình → sinh userflow-{group}.md
/vnr-qckit:qc_map_flow	Mapping	Sinh {AliasPage}.ts + {AliasLocator}.ts + locator-map.md
/vnr-qckit:qc_url_page_map	Mapping	Khớp alias → Page Object đã có → sinh url-page-map.md
/vnr-qckit:qc_sub-system-map	Mapping	Trích xuất màn hình → group → sinh sub-system-map.json
/vnr-qckit:qc_generate	Sinh test	Sinh testcase.md, datafake.json, knowledge/*.md từ spec PBI
/vnr-qckit:qc_auto_test	Chạy test	Chạy TC Auto chưa PASS, cập nhật kết quả vào testcase.md
/vnr-qckit:qc_triage	Phân tích	Phân loại PASS/APP_BUG/INFRA, sinh bảng Severity
/vnr-qckit:playwright_cli	Driver	Điều khiển browser qua MCP (dùng bởi các skill khác)

Luồng thực thi theo giai đoạn

GIAI ĐOẠN 1 – CÀI ĐẶT & KHỞI TẠO (chạy 1 lần khi init project)

/vnr-qckit:qc_pre

- Bước 1 → /vnr-qckit:qc_setup
 - Tạo .specify/tests/ + thư mục con
 - npm install @playwright/test
 - Copy Chromium → %LOCALAPPDATA%\pw-chromium\
 - Tạo tsconfig.json
 - Kiểm tra/tạo playwright.config.ts
(readEnvFile, headless từ env, REPORT_DIR)
 - Kiểm tra/tạo global-setup.ts
- CHECK 2 → Kiểm tra BasePage.ts
 - Thiếu → /vnr-qckit:qc_basepage
 - Ghi BasePage.ts từ template
 - Verify methods bằng Playwright
- CHECK 3 → Crawl navigation từ APP_URL
 - Sinh/cập nhật url-aliases.md



GIAI ĐOẠN 2 – KHÁM PHÁ ỨNG DỤNG

/vnr-qckit:qc_detect_component [group|alias|--all]

- Đọc url-aliases.md + component-rule.md + session
- Mở từng URL, crawl DOM (tối đa 30s/màn hình)
- Sinh .specify/memory/components/component_{alias}.md

/vnr-qckit:qc_component_rule (chạy sau detect_component)

- Đọc tất cả component_{alias}.md
- Tổng hợp component types mới chưa có rule
- Append vào .specify/rules/component-rule.md

/vnr-qckit:qc_user_flow [alias|group|--all]

- Crawl action + form fields trên từng màn hình (60s/màn hình)
- Sinh ASCII flow diagram: Group→Alias→Feature→Screen→Action
- Ghi .specify/memory/userflow-{group}.md



GIAI ĐOẠN 3 – MAPPING PAGE OBJECT

/vnr-qckit:qc_map_flow [alias|--all|--pbi PBI_ID]

- Đọc component_temp_{alias}.md + component-rule.md + userflow
- Sinh .specify/tests/pages/{group}/{AliasName}Locator.ts
- Sinh .specify/tests/pages/{group}/{alias}/{AliasName}Page.ts
- Sinh .specify/tests/knowledge/{group}/{alias}/locator-map.md

```

/vnr-qckit:qc_url_page_map
├─ Glob *Page.ts + *Locator.ts trong pages/
├─ Khớp alias → Page class theo score >= 0.6
└─ Sinh .specify/tests/pages/url-page-map.md

/vnr-qckit:qc_sub-system-map
├─ Parse bằng "Đã có Page Object" trong url-page-map.md
└─ Sinh .specify/memory/sub-system-map.json

```



GIAI ĐOẠN 4 – SINH TEST CASE (per PBI)

```

/vnr-qckit:qc_generate <PBI_ID>
├─ Đọc song song: spec.md, plan.md, userflow, domain-knowledge,
│  mockup ảnh, data-catalog, sub-system-map.json
├─ Phân tích: màn hình, fields, business rules, nhóm TC
├─ Sinh .specify/specs/{PBI_ID}/testcase.md
├─ Sinh .specify/specs/{PBI_ID}/datafake.json
└─ Sinh .specify/tests/knowledge/{PBI_ID}/screen-summary.md
    + field-catalog.md

```



GIAI ĐOẠN 5 – CHẠY TEST TỰ ĐỘNG (per PBI, lặp lại)

```

/vnr-qckit:qc_auto_test <PBI_ID>
├─ Đọc testcase.md → lọc TC Auto chưa PASSED (Mục 4)
├─ Bóc tách bước → Page Object/Locator
├─ Sinh/tái sử dụng TC-{NNN}.spec.ts + {PBI_ID}.spec.ts
├─ Chạy playwright-cli (execution) + playwright-mcp (discovery)
├─ Cập nhật PageObject/Locator nếu phát hiện component mới
├─ Ghi kết quả vào testcase.md (PASSED / FAILED / SKIPPED)
└─ Ghi HISTORY.md + run-summary.md

```



GIAI ĐOẠN 6 – PHÂN LOẠI KẾT QUẢ

```

/vnr-qckit:qc_triage <PBI_ID>
├─ Đọc HISTORY.md → lấy run mới nhất
├─ Đọc run-summary.md + testcase.md
├─ Phân loại từng TC:  PASS /  APP_BUG /  INFRA
├─ Xác định Severity:  Critical /  High /  Medium /  Low
└─ Sinh auto_test_results/triage-{PBI_ID}-{YYYYMMDD}.md

```



Fix lỗi → lặp lại GD 5 & 6

Chi tiết từng skill

Giai đoạn 1 — Cài đặt & Khởi tạo

Skill: qc_pre (Orchestrator)

Lệnh duy nhất cần gọi khi khởi tạo project mới. Tự động gọi qc_setup và qc_basepage.

Gọi bằng	/vnr-qckit:qc_pre
Arguments	(không có)
Mục đích	Orchestrate 3 bước: setup config → tạo BasePage → crawl URL

Input:

File	Bắt buộc	Ghi chú
.specify/tests/.env	✓	Chứa ENV=local (hoặc tên env khác)
.specify/tests/.env.{ENV}	✓	Chứa APP_URL=http://...
.specify/tests/url-aliases.md	—	Đọc nếu đã tồn tại để giữ alias cũ

Output:

File	Trạng thái	Sinh bởi bước
.specify/tests/playwright.config.ts	Tạo/sửa	Bước 1 (qc_setup)
.specify/tests/global-setup.ts	Tạo nếu thiếu	Bước 1 (qc_setup)
.specify/tests/tsconfig.json	Tạo nếu thiếu	Bước 1 (qc_setup)
%LOCALAPPDATA%\pw-chromium\chrome-win\	Copy	Bước 1 (qc_setup)
.specify/tests/pages/BasePage.ts	Tạo nếu thiếu	CHECK 2 (qc_basepage)
.specify/tests/url-aliases.md	Tạo / cập nhật	CHECK 3 (crawl navigation)

Luồng xử lý:

Bước 1 → Gọi /vnr-qckit:qc_setup (chờ hoàn thành)
CHECK 2 → Kiểm tra BasePage.ts; thiếu → gọi /vnr-qckit:qc_basepage
CHECK 3 → Đọc APP_URL từ .env → crawl navigation → ghi url-aliases.md

Báo cáo kết quả:

```
[v] qc_pre hoàn thành
Playwright + Chromium : [v] đã cài
playwright.config.ts   : OK | [edit] đã sửa | [new] đã tạo
global-setup.ts        : OK | [new] đã tạo
BasePage.ts            : EXISTS | CREATED | UPDATED | FAILED
url-aliases.md         : added={n}, updated={n}, unchanged={n}, warning={n}
```

Skill: qc_setup (Internal)

Được gọi tự động bởi `qc_pre`. Có thể chạy độc lập để reset cấu hình.

Gọi bằng	<code>/vnr-qckit:qc_setup</code> hoặc tự động từ <code>qc_pre</code>
Arguments	(không có)
Mục đích	Cài Playwright + Chromium, kiểm tra/tạo playwright.config.ts và global-setup.ts

Input:

File/Thư mục	Bắt buộc	Ghi chú
<code>.specify/tests/.env</code>	✓	Phải tồn tại trước khi chạy
<code>{PLUGIN_DIR}\tools\pw-chromium\chrome-win\</code>	✓	Chromium bundled sẵn trong plugin

Output:

File	Trạng thái	Ghi chú
<code>.specify/tests/</code> (thư mục + node_modules)	Tạo + cài	<code>@playwright/test</code> được npm install
<code>.specify/tests/tsconfig.json</code>	Tạo nếu thiếu	<code>ES2020, commonjs, strict: true</code>
<code>.specify/tests/playwright.config.ts</code>	Tạo/sửa	headless từ env, readEnvFile, REPORT_DIR, executablePath
<code>.specify/tests/global-setup.ts</code>	Tạo nếu thiếu	Session login (tự reuse/refresh user.json)
<code>%LOCALAPPDATA%\pw-chromium\chrome-win\</code>	Copy	Chromium cố định, dùng làm executablePath

Luồng xử lý (6 CHECK tuần tự):

```

CHECK 0 → Tạo .specify/tests/ + runs/ + pages/
        npm install @playwright/test --prefix .specify/tests
        Copy Chromium → %LOCALAPPDATA%\pw-chromium\chrome-win\
        Tạo tsconfig.json nếu thiếu

CHECK 1 → playwright.config.ts tồn tại?
        MISSING → tạo mới với chuẩn (readEnvFile, headless, executablePath,
        REPORT_DIR)
        OK → tiếp CHECK 2

CHECK 2 → readEnvFile có trong config?
        MISSING → chèn hàm readEnvFile + đọc baseUrl từ .env
        OK → tiếp CHECK 3

CHECK 3 → headless đọc từ env (PW_HEADLESS)?
        MISSING → sửa thành: (process.env.PW_HEADLESS ?? envVars.PW_HEADLESS ??
        'true') !== 'false'

```

OK → tiếp CHECK 4

CHECK 4 → REPORT_DIR có trong config?

MISSING → thêm biến reportDir = process.env.REPORT_DIR || timestamp

OK → tiếp CHECK 5

CHECK 5 → global-setup.ts tồn tại?

MISSING → ghi từ template chuẩn

OK → báo cáo kết quả

CHECK 6 → BasePage.navigate() dùng _resolveAppUrl()?

MISSING/OUTDATED → edit BasePage.ts: thêm _resolveAppUrl(), sửa navigate()

OK → báo cáo kết quả

Skill: `qc_basepage` (*Internal*)

Được gọi tự động bởi `qc_pre`. Có thể chạy độc lập khi cần cập nhật BasePage.

Gọi bằng `/vnr-qckit:qc_basepage` hoặc tự động từ `qc_pre`

Arguments (không có)

Mục đích Tạo/cập nhật BasePage.ts — lớp cha chứa common methods cho toàn bộ Page Object

Input:

File	Bắt buộc	Ghi chú
<code>.specify/tests/pages/BasePage.ts</code>	—	Đọc nếu tồn tại để kiểm tra trước khi ghi đè
<code>.specify/tests/.env + .env.{ENV}</code>	✓	Lấy APP_URL để verify methods

Output:

File	Trạng thái	Ghi chú
<code>.specify/tests/pages/BasePage.ts</code>	Tạo mới / giữ nguyên	Template chuẩn với <code>_resolveAppUrl()</code>
<code>.specify/tests/tmp/_basepage_verify.spec.ts</code>	Tạm (tự xóa sau)	Test tạm để verify từng method

Methods trong BasePage.ts:

<code>navigate(url)</code>	– resolve <code>_APP_URL</code> + hash routing, goto + domcontentloaded
<code>handleConfirmDialog(btn)</code>	– click confirm dialog nếu xuất hiện, no-op nếu không có
<code>waitForToast(text)</code>	– chờ toast/notification chứa text
<code>verifyUrl(pattern)</code>	– assert URL khớp string/RegExp
<code>inputTextbox(loc, val)</code>	– click + fill textbox
<code>verifyTextbox(loc, exp)</code>	– expect(locator).toHaveValue(expected)
<code>inputCombobox(loc, val)</code>	– fill + chờ dropdown jQuery autocomplete + click item
<code>inputCheckbox(loc, checked)</code>	– tích/bỏ tích nếu trạng thái hiện tại khác yêu cầu

```
verifyCheckbox(loc, exp)    – expect checked/not.toBeChecked()
inputTable(tbody, addRow, rows, columnMap) – nhập dữ liệu theo danh sách TableRow[]
```

Luồng xử lý (5 Bước):

Bước 1 → Kiểm tra BasePage.ts đã tồn tại?
 False → tạo từ template chuẩn
 True → đọc, không ghi đề nếu không có yêu cầu sửa

Bước 2 → Ghi BasePage.ts từ template (khi thiếu)

Bước 3 → Tạo test tạm .specify/tests/tmp/_basepage_verify.spec.ts
 → chạy: `npx playwright test tmp/_basepage_verify.spec.ts --workers=1`

Bước 4 → Method fail? Chỉ được sửa: selector, timeout, logic nội bộ
 KHÔNG được sửa: tên method, số tham số, kiểu trả về, signature

Bước 5 → Xóa file test tạm → báo cáo: Pass / Skip / FailFix / Failed

Giai đoạn 2 — Khám phá Ứng dụng

Skill: `qc_detect_component`

Gọi bằng	<code>/vnr-qckit:qc_detect_component [group-code alias1 alias2 --all]</code>
Mục đích	Crawl DOM từng màn hình, phát hiện component theo rule và element chưa có rule
Tools	<code>mcp__playwright__*</code> (ưu tiên) → <code>playwright-cli</code> → <code>npx playwright</code> (fallback)

Input:

File	Bắt buộc	Ghi chú
<code>.specify/tests/url-aliases.md</code>	✓	Danh sách alias + path cần crawl
<code>.specify/rules/component-rule.md</code>	✓	Selector rule cho từng component type
<code>.specify/tests/user.json</code>	✓	Session login; tự refresh nếu hết hạn
<code>.specify/tests/.env</code> + <code>.env.{ENV}</code>	✓	APP_URL, AUTH_USERNAME, AUTH_PASSWORD

Output:

File	Trạng thái	Ghi chú
<code>.specify/memory/components/component_{alias}.md</code>	Tạo mới / ghi đè	Một file per màn hình

Format output `component_{alias}.md`:

```
# Component List – {alias}
> URL: {path} | Sinh: {ISO_DATE}

## {ComponentType} – {mô tả} ({n} fields)
| # | Label | formControlName | Placeholder | Required | Context |
|---|---|---|---|---|---|
| 1 | Tên phiếu | tenPhieu | Nhập tên... | ✓ | dialog |
```

Luồng xử lý:

```
BƯỚC 0 → Đọc .env → APP_URL; kiểm tra url-aliases.md + component-rule.md
        Nạp session user.json vào playwright-mcp
        Parse argument → danh sách alias cần xử lý

BƯỚC 1 → Với từng alias (tối đa 30s/màn hình):
        - Mở URL → wait networkidle
        - Detect component tags theo rule trong component-rule.md
        - Thu thập: label, formControlName, placeholder, required, context
        - Ghi component_{alias}.md

BƯỚC 2 → Báo cáo: tổng màn hình xử lý, component tìm thấy, component mới
```

Skill: qc_component_rule

Chạy sau qc_detect_component khi phát hiện component type mới.

Gọi bằng	/vnr-qckit:qc_component_rule
Arguments	(không có)
Mục đích	Tổng hợp component types mới chưa có rule, cập nhật component-rule.md

Input:

File	Bắt buộc	Ghi chú
.specify/memory/components/component_*.md	✓	Ít nhất 1 file; thiếu → DỪNG
.specify/rules/component-rule.md	—	Đọc nếu tồn tại để biết type nào đã có rule
{PLUGIN_DIR}/templates/template-component-rule.md	—	Template format output
{PLUGIN_DIR}/prompts/components.md	—	DOM HTML reference Angular components
{PLUGIN_DIR}/prompts/components_core.md	—	DOM HTML reference Kendo/legacy

Output:

File	Trạng thái	Ghi chú
<code>.specify/rules/component-rule.md</code>	Tạo mới / append	Định nghĩa container, label, input selector cho mỗi type

Luồng xử lý:

BƯỚC 0 → Quét `.specify/memory/components/component_*.md`
Không tìm thấy file nào → DỪNG

BƯỚC 1 → Đọc đồng thời: `component-rule.md` (file đích) + reference DOM files + template

BƯỚC 2 → Tổng hợp unique component types từ tất cả `component_*.md`
→ Lọc ra các type chưa có trong `component-rule.md`
→ Ánh xạ mỗi type → DOM HTML từ `components.md` / `components_core.md`

BƯỚC 3 → Với type đã có rule: hỏi user ghi đè hay giữ nguyên

BƯỚC 4 → Append entry mới vào `component-rule.md` theo format chuẩn

BƯỚC 5 → Báo cáo: CREATED / UPDATED / SKIPPED / TODO

Skill: qc_user_flow

Gọi bằng	<code>/vnr-qckit:qc_user_flow [alias1 alias2 group-code --all]</code>
Mục đích	Crawl user flow thực tế, sinh ASCII diagram, ghi <code>userflow-{group}.md</code>
Tools	<code>mcp__playwright__*</code> (ưu tiên) → <code>playwright-cli</code> → <code>npx playwright</code> (fallback)

Input:

File	Bắt buộc	Ghi chú
<code>.specify/tests/url-aliases.md</code>	✓	Danh sách alias + path cần crawl
<code>.specify/tests/user.json</code>	✓	Session login hợp lệ
<code>.specify/tests/.env</code> + <code>.env.{ENV}</code>	✓	APP_URL
<code>.specify/memory/flow-index.md</code>	—	Cập nhật entry nếu tồn tại

Output:

File	Trạng thái	Ghi chú
<code>.specify/memory/userflow-{group}.md</code>	Tạo / cập nhật	ASCII flow diagram + mô tả màn hình
<code>.specify/memory/flow-index.md</code>	Cập nhật	Index thời gian + trạng thái URL
<code>.specify/tests/url-aliases.md</code>	Cập nhật nếu phát hiện URL mới	Thêm alias <code>[new]</code>

Format output `userflow-{group}.md`:

```

Group → Alias → Feature → Screen → Action
ATT
att_leave_day – DS Ngày nghỉ
Toolbar: [Thêm mới: open-dialog] [Xuất Excel: export] [Xóa: delete]
Happy path – Tạo mới:
1. Navigate att_leave_day
2. Click Thêm mới → dialog mở
3. inputCombobox(loaiNgayNghỉ, "...")
4. Click Lưu → toast "Thêm mới thành công"

```

Luồng xử lý:

```

BƯỚC 0 → Kiểm tra env, url-aliases.md, session
Parse argument → danh sách alias

BƯỚC 1 → Crawl từng màn hình (tối đa 60s/màn hình):
- Mở URL, chụp snapshot (mcp__playwright__browser_snapshot)
- Phân loại: List / Form / Detail / Tab / Dashboard
- Crawl toolbar actions (label, type, state, disabled?)
- Crawl form fields (label, type, required, validation, options)
- Crawl sub-tabs nếu có
- Crawl pagination, filter, search

BƯỚC 2 → Chuẩn hoá:
- Suy luận happy path, luồng phụ
- Map field_type → BasePage method
- Sinh ASCII flow diagram

BƯỚC 3 → Ghi userflow-{group}.md
BƯỚC 4 → Cập nhật url-aliases.md (nếu phát hiện URL mới)
BƯỚC 5 → Cập nhật flow-index.md

```

Giai đoạn 3 — Mapping Page Object**Skill: `qc_map_flow`**

Gọi bằng `/vnr-qckit:qc_map_flow [alias1 alias2 | --all | --pbi <PBI_ID>]`

Mục đích Sinh {AliasName}Locator.ts + {AliasName}Page.ts từ component_temp + userflow

Input:

File	Bắt buộc	Ghi chú
<code>vnr-qckit/detected_components/component_temp_{alias}.md</code>		Thiếu → DỪNG, nhắc chạy <code>qc_detect_component</code>

File	Bắt buộc	Ghi chú
<code>.specify/tests/url-aliases.md</code>	✓	Alias → path + group detection
<code>.specify/rules/component-rule.md</code>	Ưu tiên	Selector rules; fallback sang template
<code>.specify/memory/userflow-{group}.md</code>	Ưu tiên	Tìm trước
<code>.specify/memory/userflow.md</code>	Fallback	Dùng khi không có file theo group
<code>.specify/specs/{PBI_ID}/testcase.md</code>	Tùy chọn	Suy luận method cần ưu tiên (<code>--pbi</code>)

Output:

File	Trạng thái	Ghi chú
<code>.specify/tests/pages/{group}/{AliasName}Locator.ts</code>	Tạo / ghi đè	Khai báo tất cả locator
<code>.specify/tests/pages/{group}/{alias}/{AliasName}Page.ts</code>	Tạo / ghi đè	Action methods, import Locator
<code>.specify/tests/knowledge/{group}/{alias}/locator-map.md</code>	Tạo / ghi đè	Tài liệu tham chiếu locator

Quy tắc đặt tên:

```
alias: att_leave_day (group: att) → AttLeaveDayLocator.ts + AttLeaveDayPage.ts
alias: cat_leave_day_type (group: cat) → CatLeaveDayTypeLocator.ts +
CatLeaveDayTypePage.ts
```

Ưu tiên locator strategy:

1. <code>getByRole('{role}', { name: '{label}' })</code>	– ARIA role (nút, link)
2. <code>getByLabel('{label}')</code>	– label rõ ràng
3. <code>locator(container).filter(hasText).locator(input)</code>	– custom component
4. <code>locator('[formcontrolname="..."]')</code>	– formControlName
5. <code>locator(':nth-child(n) input')</code>	– fallback index

Luồng xử lý:

```
BƯỚC 0 → Xác định alias list + group theo argument
          Kiểm tra file nguồn; thiếu → DỪNG

BƯỚC 1 → Đọc song song: url-aliases.md, component-rule.md, userflow file
          Xây: ComponentRoleMap + FieldInventory + ActionMap

BƯỚC 2 → Tính locator strategy tối ưu cho từng field/button
```

BƯỚC 3 → Sinh {AliasName}Locator.ts (khai báo locators)
Sinh {AliasName}Page.ts (action methods, import Locator)

BƯỚC 4 → Sinh locator-map.md tài liệu tham chiếu

BƯỚC 5 → Cập nhật _IMPACT_INDEX.json + flow-index.md nếu có

Skill: qc_url_page_map

Gọi bằng /vnr-qckit:qc_url_page_map

Arguments (không có)

Mục đích Quét Page Object đã có, khớp alias → Page class, sinh url-page-map.md

Input:

File	Bắt buộc	Ghi chú
.specify/tests/url-aliases.md	✓	Danh sách alias + path
.specify/tests/pages/**/*Page.ts	—	Các Page Object đã sinh
.specify/tests/pages/**/*Locator.ts	—	Các Locator đã sinh

Output:

File	Trạng thái	Ghi chú
.specify/tests/pages/url-page-map.md	Tạo / ghi đè	Bảng: đã khớp + chưa khớp + thống kê

Thuật toán khớp (score ≥ 0.6):

alias: att_leave_day → segments normalize → "attleaveday"
Page class: AttLeaveDayPage → normalize → "attleaveday"
Score = ký tự khớp liên tiếp / độ dài chuỗi ngắn hơn

Luồng xử lý:

BƯỚC 0 → Kiểm tra url-aliases.md + thư mục pages/
BƯỚC 1 → Glob *Page.ts + *Locator.ts → extract ClassName + path
Ghép cặp Page ↔ Locator theo prefix
BƯỚC 2 → Parse url-aliases.md → alias + path + section
BƯỚC 3 → Tính score khớp mỗi alias × Page class; chọn score cao nhất ≥ 0.6
BƯỚC 4 → Sinh url-page-map.md (đã khớp / chưa khớp / thống kê theo section)
BƯỚC 5 → Báo cáo: tổng alias, đã khớp, chưa có PO

Skill: qc_sub-system-map

Gọi bằng	<code>/vnr-qckit:qc_sub-system-map</code>
Arguments	<i>(không có)</i>
Mục đích	Trích xuất mapping màn hình → group, sinh sub-system-map.json dùng cho qc_generate

Input:

File	Bắt buộc	Ghi chú
<code>.specify/tests/pages/url-page-map.md</code>	✓	Chỉ đọc section "Đã có Page Object"

Output:

File	Trạng thái	Ghi chú
<code>.specify/memory/sub-system-map.json</code>	Tạo / ghi đè	JSON: [{ <code>ma_man_hinh</code> , <code>ten_man_hinh</code> , <code>group</code> }]

Quy tắc trích xuất `group` và `ma_man_hinh`:

```

Page File: att/att_leave_day/AttLeaveDayPage.ts
→ group      = "att"          (segment[0])
→ ma_man_hinh = "att_leave_day" (segment[1])

Page File: hre/HreProfilePage.ts
→ group      = "hre"
→ ma_man_hinh = "HreProfile"  (fallback: tên file bỏ suffix Page)

```

Luồng xử lý:

```

BƯỚC 0 → Kiểm tra url-page-map.md; thiếu → DỪNG
BƯỚC 1 → Parse bảng "Đã có Page Object"
BƯỚC 2 → Trích xuất group + ma_man_hinh từ cột Page File
BƯỚC 3 → Deduplicate theo page_file (giữ dòng đầu)
BƯỚC 4 → Ghi sub-system-map.json (sort: group ASC, ma_man_hinh ASC)
BƯỚC 5 → Báo cáo số lượng

```

Giai đoạn 4 — Sinh Test Case**Skill: `qc_generate`**

Gọi bằng	<code>/vnr-qckit:qc_generate <PBI_ID></code>
Arguments	<code>PBI_ID</code> — bắt buộc
Mục đích	Sinh testcase.md, datafake.json, knowledge/*.md từ spec + plan + flow

Input:

File	Bắt buộc	Ghi chú
<code>.specify/specs/{PBI_ID}/spec.md</code>	✓	Thiếu → DỪNG ngay
<code>.specify/specs/{PBI_ID}/plan.md</code>	✓	Thiếu → DỪNG ngay
<code>.specify/memory/userflow-{group}.md</code>	✓	Thiếu → DỪNG, hỏi group màn hình
<code>.specify/templates/testcase_template.md</code>	—	Template chuẩn cho testcase.md (ưu tiên dùng)
<code>.specify/memory/constitution.md</code>	—	Quy tắc dự án
<code>.specify/memory/domain-knowledge.md</code>	—	Business domain
<code>.specify/specs/{PBI_ID}/assets/*.png,*.jpg</code>	—	UI mockup (đọc qua vision)
<code>.specify/tests/data-catalog/categories/</code>	—	Dữ liệu thực (vendors, customers, ...)
<code>.specify/memory/sub-system-map.json</code>	—	Kiểm tra Group/Màn hình hợp lệ
<code>.specify/tests/knowledge/_IMPACT_INDEX.json</code>	—	Load business + impact knowledge

Output:

File	Trạng thái	Ghi chú
<code>.specify/specs/{PBI_ID}/testcase.md</code>	Tạo mới	Danh sách TC đầy đủ (Mục 1–5)
<code>.specify/specs/{PBI_ID}/datafake.json</code>	Tạo mới	Data giả có nghĩa nghiệp vụ
<code>.specify/tests/knowledge/{PBI_ID}/screen-summary.md</code>	Tạo mới	Mô tả màn hình, luồng, business rules
<code>.specify/tests/knowledge/{PBI_ID}/field-catalog.md</code>	Tạo mới	Danh mục field và validation
<code>.specify/tests/knowledge/_IMPACT_INDEX.json</code>	Cập nhật nếu có	Thêm entry PBI_ID

Nhóm test case sinh ra:

- P1 – Luồng chính (happy path): CRUD cơ bản
- P1 – Validation: required fields, format, boundary
- P1 – Business rule: logic nghiệp vụ đặc thù
- P2 – Edge case: empty state, error response
- P2 – Permission: phân quyền theo role

Phân loại Loại TC:

- ✓ Auto – Playwright có thể thực thi hoàn toàn
- ⚠ Semi-Auto – Playwright thực thi một phần, cần confirm thủ công
- 👉 Manual – Playwright không thể test (captcha, email, file upload)

Luồng xử lý (6 Bước):

Bước 1 → Đọc song song tất cả tài liệu

Bước 2 → Phân tích spec: màn hình, fields, business rules, nhóm TC

Bước 3 → Sinh testcase.md (theo template nếu có)

Bước 4 → Sinh datafake.json (happy_path, validation, edge_cases)
Quy tắc: KHÔNG dùng foo/bar/test123; ưu tiên data-catalog → domain → format VN

Bước 5 → Sinh knowledge/*.md (screen-summary, field-catalog)

Bước 6 → Cập nhật _IMPACT_INDEX.json nếu tồn tại

Giai đoạn 5 — Chạy Test Tự động

Skill: qc_auto_test

Gọi bằng	/vnr-qckit:qc_auto_test <PBI_ID>
Arguments	<PBI_ID> (bắt buộc) / --reset TC-001,TC-002 / --reset-all / --uutien TC-001
Mục đích	Chạy TC Auto chưa PASSED, cập nhật kết quả vào testcase.md Mục 4
Tools	playwright-cli (execution) + playwright-mcp (discovery DOM khi cần)

Phân công tool:

playwright-cli → MỌI thao tác thực thi:
navigate, fill, click, check, assert_visible/text/url/value, run_test

playwright-mcp → CHỈ dùng cho discovery:
browser_snapshot (tìm selector mới)
browser_click + browser_type + browser_snapshot (Kendo combobox popup)
browser_current_url (kiểm tra redirect auth guard)

Input:

File	Bắt buộc	Ghi chú
.specify/specs/{PBI_ID}/testcase.md	✓	Mục 4 — nguồn TC + kết quả; thiếu → DỪNG
.specify/specs/{PBI_ID}/datafake.json	✓	Dữ liệu test; thiếu → DỪNG
.specify/tests/pages/BasePage.ts	✓	Thiếu → nhắc chạy /qc_basepage
.specify/tests/user.json	✓	Session login
.specify/tests/.env + .env.{ENV}	✓	APP_URL
.specify/rules/component-rule.md	—	Selector hints khi gặp component mới
auto_test_results/TC-{NNN}.spec.ts	—	Tái sử dụng nếu bước TC không đổi

Output:

File	Trạng thái	Ghi chú
<code>.specify/specs/{PBI_ID}/testcase.md</code>	Cập nhật Mục 4	Thêm Kết quả thực tế + Chạy lúc
<code>auto_test_results/{PBI_ID}.spec.ts</code>	Tạo / ghi đè	Script batch (ghi đè mỗi lần chạy)
<code>auto_test_results/TC-{NNN}.spec.ts</code>	Tạo / tái dùng	Script per-TC (tái sử dụng nếu bước không đổi)
<code>auto_test_results/run-{DATETIME}.json</code>	Tạo mới	Playwright raw output
<code>auto_test_results/run-{DATETIME}-summary.md</code>	Tạo mới	Tóm tắt lần chạy
<code>auto_test_results/HISTORY.md</code>	Append	Lịch sử tất cả lần chạy
<code>.specify/tests/runs/{PBI_ID}.spec.ts</code>	Tạm	Copy để Playwright chạy

Logic tái sử dụng script:

TC-{NNN}.spec.ts đã có + bước TC KHÔNG đổi → GIỮ NGUYÊN (không sinh lại)
 TC-{NNN}.spec.ts đã có + bước TC ĐÃ ĐỔI → SINH LẠI

Quy tắc quan trọng:

TC có  PASSED → KHÔNG chạy lại (trừ --reset)
 TC có  FAILED /  SKIPPED → chạy lại lần tiếp theo
 Chỉ chạy TC Loại =  Auto, bỏ qua Semi-Auto và Manual
 Thứ tự ưu tiên chạy: --uutien → p0 → p1 → p2 → ...

Luồng xử lý (8 Bước):

Bước 0 → Đọc .env → APP_URL
 Bước 1 → Đọc testcase.md Mục 4 → lọc TC Auto chưa PASSED
 Bước 2 → Đọc datafake.json + BasePage.ts + component-rule.md
 Bước 3 → Bóc tách bước từng TC → xác định Page Object + Locator cần dùng
 Kiểm tra script tái sử dụng; sinh/cập nhật TC-NNN.spec.ts + batch spec
 Bước 4 → playwright-cli load_storage_state user.json
 Copy spec → .specify/tests/runs/
 Chạy: playwright-cli run_test hoặc npx playwright test
 Bước 5 → Parse output → PASSED / FAILED / SKIPPED từng TC
 Nếu FAILED: playwright-mcp browser_snapshot để discovery DOM nếu cần
 Bước 6 → Cập nhật testcase.md Mục 4 (Kết quả thực tế + Chạy lúc)
 Bước 7 → Ghi run-{DATETIME}-summary.md + append HISTORY.md
 Bước 8 → Cập nhật bảng Tóm tắt trong testcase.md (cột Đã pass)

Skill: qc_triage

Gọi bằng	/vnr-qckit:qc_triage <PBI_ID>
Arguments	PBI_ID — bắt buộc
Mục đích	Phân loại kết quả PASS/APP_BUG/INFRA, xác định Severity, sinh báo cáo triage

Input:

File	Bắt buộc	Ghi chú
auto_test_results/HISTORY.md	✓	Thiếu → DỪNG (nhắc chạy qc_auto_test trước)
auto_test_results/run-{DATETIME}-summary.md	Ưu tiên	Summary của run mới nhất
.specify/specs/{PBI_ID}/testcase.md	✓	Lấy mô tả, nhóm, priority, kết quả inline

Output:

File	Trạng thái	Ghi chú
auto_test_results/triage-{PBI_ID}-{YYYYMMDD}.md	Tạo / ghi đè	Báo cáo phân loại đầy đủ + hành động tiếp theo

3 loại phân loại:

Loại	Ký hiệu	Định nghĩa
PASS	✓	Test exit 0, tất cả assertion green
APP_BUG	🐛	App sai nghiệp vụ: feature missing, validation sai, business rule sai, UI sai spec
INFRA	🔧	Lỗi môi trường: session expired, host mismatch, selector stale, network timeout, script bug

Bảng quyết định phân loại:

TC FAILED →		
├─	TimeoutError + redirect/login/cookie mismatch	→ INFRA (session)
├─	TimeoutError + selector không match + FE chưa deploy	→ APP_BUG (feature missing)
├─	TimeoutError + selector stale sau FE refactor	→ INFRA (stale selector)
├─	Assertion fail + app không block khi phải block	→ APP_BUG (validation missing)
├─	Assertion fail + message sai	→ APP_BUG (wrong message)
├─	TypeError / undefined trong script	→ INFRA (script bug)
├─	networkidle timeout + server không phản hồi	→ INFRA (network/env)
└─	Không rõ nguyên nhân	→ APP_BUG (cần dev xác nhận)
TC PASSED → ✓ PASS		

Severity cho APP_BUG:

Severity	Điều kiện
Critical	Feature core P1 không hoạt động, hoặc security/permission bị vi phạm
High	Business rule P1 sai, validation bắt buộc bị thiếu, filter hiển thị data sai
Medium	Message/toast sai nội dung, field không persist, conditional show/hide sai
Low	UI cosmetic, label sai, format ngày sai, default value sai (không ảnh hưởng save)

Luồng xử lý (5 Bước):

Bước 1 → Kiểm tra HISTORY.md → xác định RUN_ID và run mới nhất
Đọc summary file + testcase.md

Bước 2 → Phân loại từng TC: PASS / APP_BUG / INFRA
Ưu tiên loại trừ INFRA trước khi kết luận APP_BUG

Bước 3 → Xác định Severity cho mỗi APP_BUG
Gom bug ảnh hưởng nhiều TC vào 1 entry

Bước 4 → Sinh triage-{PBI_ID}-{YYYYMMDD}.md:
- Bảng tổng quan (PASS/APP_BUG/INFRA + %)
- Danh sách PASS
- Bảng APP_BUG theo Severity + chi tiết từng bug
- Bảng INFRA + action fix
- Hành động tiếp theo ưu tiên (INFRA fix trước)

Bước 5 → Báo cáo console: tổng quan + bug nổi bật + bước tiếp theo

Bảng Input/Output nhanh

Skill	Input chính	Output chính	Ghi đề?
qc_setup	.env, plugin Chromium	playwright.config.ts, global-setup.ts, tsconfig.json	Tạo/sửa
qc_basepage	template hardcode	BasePage.ts	Tạo nếu thiếu
qc_pre	.env, app đang chạy	url-aliases.md + gọi qc_setup + qc_basepage	Cập nhật
qc_detect_component	url-aliases.md, component-rule.md, session	components/component_{alias}.md	Ghi đề
qc_component_rule	component_*.md, DOM reference	component-rule.md	Append
qc_user_flow	url-aliases.md, session	userflow-{group}.md, flow-index.md	Cập nhật
qc_map_flow	component_temp_{alias}.md, component-rule.md,	{AliasLocator}.ts, {AliasPage}.ts, locator-map.md	Ghi đề

Skill	Input chính	Output chính	Ghi đề?
	userflow.md		
qc_url_page_map	url-aliases.md, *Page.ts, *Locator.ts	url-page-map.md	Ghi đề
qc_sub-system-map	url-page-map.md	sub-system-map.json	Ghi đề
qc_generate	spec.md, plan.md, userflow-*.md	testcase.md, datafake.json, knowledge/*.md	Tạo mới
qc_auto_test	testcase.md, datafake.json, BasePage.ts, session	testcase.md (cập nhật), *.spec.ts, HISTORY.md	Cập nhật/Append
qc_triage	HISTORY.md, run-summary.md, testcase.md	triage-{PBI_ID}-{YYYYMMDD}.md	Ghi đề

Cấu trúc thư mục output

.specify/ tests/ .env .env.local user.json playwright.config.ts global-setup.ts tsconfig.json url-aliases.md runs/ {PBI_ID}.spec.ts pages/ BasePage.ts url-page-map.md {group}/ {AliasName}Locator.ts {alias}/ {AliasName}Page.ts data-catalog/ categories/ knowledge/ _IMPACT_INDEX.json {PBI_ID}/ screen-summary.md field-catalog.md {group}/{alias}/ locator-map.md specs/ {PBI_ID}/ spec.md plan.md assets/*.png testcase.md datafake.json auto_test_results/	← INPUT: ENV=local ← INPUT: APP_URL=..., PW_HEADLESS=false ← Session state (sinh bởi global-setup.ts) ← [qc_setup] cấu hình Playwright ← [qc_setup] login + lưu user.json ← [qc_setup] TypeScript config ← [qc_pre] alias → path từ crawl navigation ← Thư mục chạy test (testDir) ← [qc_auto_test] copy tạm để Playwright chạy ← [qc_basepage] lớp cha common methods ← [qc_url_page_map] alias → Page Object ← [qc_map_flow] khai báo locators ← [qc_map_flow] action methods ← INPUT cho qc_generate ← [qc_generate] cập nhật per PBI ← [qc_generate] ← [qc_generate] ← [qc_map_flow] ← INPUT bắt buộc cho qc_generate ← INPUT bắt buộc cho qc_generate ← INPUT optional (UI mockup) ← [qc_generate] sinh; [qc_auto_test] cập nhật ← [qc_generate]
--	---

└─ {PBI_ID}.spec.ts	← [qc_auto_test] batch, ghi đề mỗi lần
└─ TC-{NNN}.spec.ts	← [qc_auto_test] per-TC, tái sử dụng
└─ run-{DATETIME}.json	← [qc_auto_test] Playwright raw output
└─ run-{DATETIME}-summary.md	← [qc_auto_test]
└─ HISTORY.md	← [qc_auto_test] lịch sử tất cả lần chạy
└─ triage-{PBI_ID}-{YYYYMMDD}.md	← [qc_triage]
memory/	
└─ constitution.md	← INPUT (quy tắc dự án)
└─ domain-knowledge.md	← INPUT (business domain)
└─ flow-index.md	← [qc_user_flow] index thời gian + trạng thái
└─ flow-{MA_PHAN_HE}.md	← INPUT cho qc_generate (flow nghiệp vụ)
└─ userflow-{group}.md	← [qc_user_flow] ASCII diagram + mô tả
└─ sub-system-map.json	← [qc_sub-system-map]
└─ components/	
└─ component_{alias}.md	← [qc_detect_component] per màn hình
rules/	
└─ component-rule.md	← [qc_component_rule] selector rules
templates/	
└─ testcase_template.md	← INPUT template cho qc_generate

Yêu cầu

- Claude Code CLI (phiên bản hỗ trợ **claude plugin**)
- Node.js >= 16
- Project có Playwright (@playwright/test)
- File **.specify/tests/.env** với **ENV=<tên_env>**
- File **.specify/tests/.env.{ENV}** với **APP_URL=http://...**
- File **.specify/tests/user.json** (sinh tự động khi chạy **global-setup.ts** lần đầu)

Quy ước

- Skill **không tự xóa** dữ liệu cũ.
- Skill **không thay đổi** **.env** nếu file đã tồn tại.
- **qc_pre** tự gọi **qc_setup** và **qc_basepage** — không cần gọi riêng khi init project.
- **qc_basepage** và **qc_setup** là internal — không chạy tự động trong pipeline ngoài **qc_pre**.
- TC đã **✅ PASSED** sẽ **không chạy lại** ở lần sau (trừ khi dùng **--reset**).
- **playwright-cli** = mọi thao tác execution; **playwright-mcp** = chỉ discovery DOM khi cần.
- Ưu tiên loại trừ INFRA **trước** khi kết luận APP_BUG trong qc_triage.

Thứ tự chạy đề xuất (project mới):

```
qc_pre → qc_detect_component → qc_component_rule → qc_user_flow
      → qc_map_flow → qc_url_page_map → qc_sub-system-map
      → qc_generate <PBI_ID> → qc_auto_test <PBI_ID> → qc_triage <PBI_ID>
```